Basic (your first real API call) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The exercise is really about seeing two different failure categories with your own eyes, not just reading about them.

A **successful call** gives you a `requests.Response` object — `.status_code: int` and `.json() -> dict | list` (it parses the body as JSON for you, raising `json.JSONDecodeError` if the body isn't valid JSON).

A **malformed URL** never reaches the network at all — `requests` raises `requests.exceptions.MissingSchema` (or a similar `requests.exceptions.RequestException` subclass) immediately, client-side, before any bytes go anywhere. This is a completely different kind of failure than a bad *response* — nothing was sent, so there's nothing to receive.

The exact pieces:

- **`requests.get(url) -> requests.Response`** — the return type is always a `Response` object on success, never `None`.
- **`requests.exceptions.RequestException`** — the base class for everything `requests` can raise client-side (bad URL, connection refused, timeout, etc.). Catching this specific base — not bare `Exception` — is the right habit once you get past this exercise.
- **`type(e).__name__`** — every exception instance knows its own class; this is how you inspect exactly what you caught without guessing.

Write both blocks — the working call and the broken one — before checking Hint 2.

Hint 2 — The plan, and almost the whole thing

```
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
print(response.json())

try:
    requests.get("htp://broken")
except requests.exceptions.RequestException as e:
    print(type(e).__name__, "-", e)
```

```
# api_first_call_practice.py
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
print(response.json())

try:
    requests.get("htp://broken")
except requests.exceptions.RequestException as e:
    ...  # what goes here?
```

Fill in the `except` block yourself, then compare against the Solution. Notice the working call needs no `try/except` at all for this exercise — you're only wrapping the call you already know will fail, to see exactly how it fails.

Full solution: Show me the solution